Лабораторна робота №1

з дисципліни «Генеративні нейронні мережі та їх застосування»

тема роботи: Використання та fine-tuning великих мовних моделей

1. **Мета роботи:** вміти розгортати та використовувати попередньо навчені LLM, проводити їх донавчання (fine-tuning) для виконання специфічних задач за допомогою бібліотеки LoRA та оцінювати якість отриманої донавченої моделі.

2. **Хід виконання роботи:**

Проаналізуйте наданий для роботи ноутбук та файли з утилітними функціями та даними з бібліотеки mitdeeplearning (надані в архіві додатково). Виконайте завдання відповідно обраному рівню складності. Під час виконання лабораторної роботи можна використовувати засоби ШІ під власну відповідальність.

3. **Звіт з лабораторної роботи** повинен містити: змінений ноутбук з результатами виконання, збережені ваги LoRA (для оцінки відмінно), висновки по кожному результату, які їх аналізують. Звіт повинен бути мінімальни, всі матеріали необхідно викласти **на власний github, з якого видно, що він ваш.**

4. **Завдання за рівнем складності** (не поділяються на варіанти):

Для всіх варіантів:

a) видалити з шаблону ноутбуку навчання та тестування роботи моделі на наборі даних leprechaun (залишити лише роботу з yoda стилем).

b) для відтворюваності результатів задати на початку ноутбуку (після імпортів) початковий seed наступним чином:

```
from transformers import set_seed

SEED = # your seed value
set_seed(SEED)

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

та про всяк випадок повторити set_seed(SEED) перед створенням dataloader та перед evaluation, щоб не боятися перезапускати окремі клітинки індивідуально (а не з повним ноутбуком)

Провести 2-3 повтори всього експерименту з різними seed. Усереднити оцінку якості моделі на основі цих результатів.

c) після відлагодження коду для фінального прогону перед відправкою на захист оцінювати якість моделі не на 5 тестових прикладах, а на 15-25 held-out prompts (слідкуйте за лімітами використання OpenRouter, бо через ліміти для фінального запуску може знадобитися окрема доба)

На оцінку «задовільно»:

1) відредагувати датасет для навчання 20-50 прикладів (можна відредагувати стиль, доповнити чи змінити стиль повністю або зовсім замінити тему)

2) змінити щонайменше 2 гіперпараметри моделі, наприклад: learning rate (1e-4 або 2e-4 або 3e-4), lora rank (4, 8, 16, 32), context length (256, 512, 1024). Порівняти і пояснити результати для різних значень гіпрепараметрів (на графіках з наданого ноутбуку).

3) показати результат до і після fine-tuning (мінімум 5 прикладів inference) з оцінкою якості суддею

На оцінку «добре»:

4) додати хоча б одну метрику якості навчання (дивіться специфіку формул у **розділі 5**): Recall, BalancedAccuracy, StyleGain

5) перемішати і розділити вибірку (дивися пояснення у **розділі 6**) навчальних прикладів на train, validation і test, задати batch_size > 10. Порівняти результати з попередніми на основі обраних метрик.

6) додати в notebook підсумкову таблицю або графік метрик для декількох (3-4) запусків процесу навчання та тестування з різними наборами параметрів. Параметри краще обирати зі списку: learning rate (1e-4 або 2e-4 або 3e-4), lora rank (4, 8, 16, 32), context length (256, 512, 1024). Порівняння треба робити, фіксуючи усі параметри, окрім одного, його змінюємо і перевіряємо, як зміна значення параметра впливає на якість моделі. Додати висновок, яка конфігурація (за якого набору значень параметрів) краща і чому.

7) показати результат до і після fine-tuning (мінімум 5 прикладів inference) з оцінкою якості суддею

На оцінку «відмінно»:

8) додати всі зазначені в **розділі 5** метрики якості навчання: Recall, BalancedAccuracy, StyleGain

9) провести *повноцінне тренування* (перемішане і з розділеною вибіркою навчальних прикладів (див. пояснення у **розділі 6**) на train, validation і test з batch_size > 10, провести декілька епох навчання) та *розширене тестування* якості моделі. Для розширеного тестування якості додати в notebook підсумкову таблицю або краще

графік значень метрик (**перелічені в розділі 5**) якості для різних (6-8) запусків процесу навчання та тестування з різними наборами параметрів.
Параметри краще обирати зі списку: learning rate(1e-4,2e-4 або 3e-4), lora rank (4, 8, 16, 32), lora_alpha (2*rank,4*rank), lora_dropout (0-0.3), context length (512, 1024), temperature при генерації тексту(0.1-1), два різні prompt templates для оцінки якості роботи моделі.

10) запропонувати та реалізувати будь-які (які вам здаються перспективними) поліпшення у навчанні та оцінці якості моделі, які покращують результати роботи. Обґрунтувати їх працездатність.

11) зберегти параметри і ваги найкращої (чи декількох) навченої моделі Lora і вміти їх завантажувати

12) продемонструвати використання найкращої навченої моделі з оцінкою якості суддею.

5. Метрики оцінки якості навчання

Позначимо

$$z_i \in \{1,0\}$$

де

- $z_i = 1$- текст згенеровано **моделлю після стилізації** (очікується стиль s)
- $z_i = 0$ - текст згенеровано **базовою моделлю** (нейтральний стиль)

Тоді

$$P = \{i: z_i = 1\}$$
$$N = \{i: z_i = 0\}$$

де

- P- стилізовані тексти
- N- нейтральні тексти

Оцінювач стилю

Нехай незалежний оцінювач (класифікатор або LLM-judge) повертає

$$q_i(s) = P(\text{style} = s \mid y_i)$$

- ймовірність того, що текст i має стиль s.

Soft-confusion matrix

Тоді:

True positive	$TP^{soft} = \sum_{i \in P} q_i\,(s)$
False Negative	$FN^{soft} = \sum_{i \in P} (1 - q_i(s))$
False Positive	$FP^{soft} = \sum_{i \in N} q_i\,(s)$
True Negative	$TN^{soft} = \sum_{i \in N} (1 - q_i(s))$

Інтерпретація:

- якщо стилізований текст має $q_i(s) = 0.9$, це майже максимальний TP
- якщо базовий (згенерований до навчання) текст має $q_i(s) = 0.4$, це частковий FP

Метрики

Soft Recall (сила стилізації після навчання**)**

$$Recall_s^{soft} = \frac{\sum_{i \in P} q_i\,(s)}{|\,P\,|}$$

Показує, **наскільки сильно стиль проявляється після навчання**.

Soft Specificity (відсутність стилю (нейтральність) у базовій моделі)

$$Specificity^{soft} = \frac{\sum_{i \in N} (1 - q_i(s))}{|\,N\,|}$$

Показує, **наскільки базова модель справді нейтральна**.

Soft Balanced Accuracy (узагальнена метрика, загальна якість керування стилем**)**

$$BalancedAccuracy^{soft} = \frac{1}{2}\left(Recall_s^{soft} + Specificity^{soft}\right)$$

Style Gain (чистий приріст стилю від навчання)

$$StyleGain = \frac{1}{|\,P\,|}\sum_{i \in P} q_i\,(s) - \frac{1}{|\,N\,|}\sum_{i \in N} q_i\,(s)$$

перший доданок показує **наскільки стиль присутній після навчання**

другий – **наскільки він був присутній до навчання**

6. **Розділення вибірки на train, validation, test**

train – це частина даних, на якій модель **навчається** (з розділенням на batch), тобто саме на цих прикладах оновлюються ваги.

епоха – це **один повний прохід моделі по всіх прикладах навчальної вибірки** (train). Перед початком кожної епохи дані з вибірки test випадково перемішуються, після чого формуються батчі, які послідовно подаються на вхід моделі під час навчання.

validation – це окрема частина, яка **не використовується для оновлення ваг**, а потрібна для контролю якості під час навчання (по завершенню одної епохи): відстеження перенавчання, вибору кращої конфігурації гіперпараметрів. Тобто **validation допомагає перевірити якість навчання та обрати модель, навчену на конкретному наборі гіперпараметрів (найкращу на думку інженера).**

test – це повністю відкладена частина, яку **не можна використовувати ні для навчання, ні для вибору гіперпараметрів**; вона потрібна тільки для чесної фінальної оцінки того, як навчена модель працює на нових даних.

Зазвичай роблять співвідношення train\validation\test як 80\10\10, але для малої навчальної вибірки (як в даній роботі) краще обирати 70\15\15.

Для реалізації такого розділення треба замість використання імпортованої з пакету **mitdeeplearning** функції **create_dataloader** розробити і використати її модифіковану реалізацію. Ця реалізація потрібно завантажувати підготовлені дані з набору databricks/databricks-dolly-15k та файлу data/text_styles/yoda.txt (дивися вихідний код поточної реалізації функції create_dataloader у наданому файлі mit_dl_utils.py в архіві mit_dl_lib.zip) перемішувати їх та ділити відповідно запропонованим пропорціям train\validation\test. При створенні вибірки train вказати бажане значення batch_size.